

# PROGRAMMA APPROFONDITO (Sintesi)

---

## 1. Fondamenti di basi di dati

Il corso introduce il ruolo strategico dei dati nei sistemi informativi moderni, illustrando come si organizzano, strutturano e gestiscono per garantire qualità, integrità, sicurezza e disponibilità. Concetti chiave:

- Dato vs informazione
  - Strutturazione, tipologie di dato
  - Ciclo di vita del dato
  - Importanza della consistenza e affidabilità
- 

## 2. Principi di elaborazione testi

Obiettivo: rendere lo studente capace di redigere documenti tecnico-scientifici professionali.

Contenuti:

- Uso avanzato del word processor (stili, titoli, indici automatici)
  - Gestione immagini, tabelle, riferimenti incrociati
  - Norme di impaginazione e leggibilità
  - Citazioni, bibliografie, note
  - Struttura logica di un report tecnico/scientifico
- 

## 3. Report tecnico-scientifici: elementi teorici e pratici

- Come progettare un report: obiettivo, destinatari, vincoli
  - Struttura tipica: abstract, metodi, analisi, risultati, discussione
  - Coerenza visiva: layout, formattazioni standardizzate, template
  - Buone pratiche di comunicazione tecnica
- 

## 4. Elementi di foglio elettronico

Introduzione all'uso di fogli di calcolo (es. Excel, Google Sheet):

- Formule di base e intermedie (SOMMA, MEDIA, SE, CERCA.VERT...)
  - Riferimenti relativi, assoluti, misti
  - Ordinamento, filtri, tabelle di dati
  - Grafici e strumenti di sintesi
- 

## 5. Fogli elettronici per il Project Management

Focus pratico orientato alla gestione dei progetti:

- Creazione di **Gantt semplificati**
  - Dashboard di progetto con indicatori (KPI)
  - Monitoraggio di tempi, costi e risorse
  - Pianificazione coerente con i **requirements del committente**
  - Analisi delle dipendenze tra attività
  - Impostazione di un **action plan** professionale
- 

## 6. Elementi di presentazione multimediale

Uso professionale di strumenti tipo PowerPoint/Google Slides:

- Struttura logico-narrativa di una presentazione
  - Coerenza grafica: palette, layout, tipografia
  - Riduzione del testo, uso efficace di visualizzazioni
  - Inserimento di grafici, diagrammi, dati, video/immagini
- 

## 7. Presentazioni per il Project Management (pitch)

Capacità di tradurre un progetto in una presentazione efficace:

- Pitch finali: struttura “problema → soluzione → impatto”
  - Presentazione dei deliverable e output finali
  - Uso di template professionali
  - Comunicazione visiva efficace, storytelling del progetto
  - Coerenza con requisiti di alto livello e risultati attesi
- 

## SEZIONE CORE: BASI DI DATI

---

## 8. Elementi di basi dati

Concetti fondamentali:

- Cos'è un database
  - Dati strutturati vs non strutturati
  - Modellazione concettuale (entità, attributi, relazioni)
  - Integrità dei dati
- 

## 9. Organizzazione dei dati in database relazionali

- Tabelle, righe, colonne
  - Tipi di dato, chiavi primarie e chiavi esterne
  - Vincoli (NOT NULL, UNIQUE, CHECK, FK)
  - Normalizzazione (1NF, 2NF, 3NF) per eliminare ridondanze e anomalie
-

## 🌐 10. Architettura delle basi di dati

L'architettura di un sistema di gestione di basi dati (DBMS) si basa su tre livelli e su un insieme di componenti che garantiscono integrità, sicurezza, efficienza e concorrenza nell'accesso ai dati.

### ➤ Architettura a tre livelli ANSI/SPARC

#### 1. Livello esterno (view level)

- Descrive come i *singoli utenti* vedono i dati.
- Comprende le *viste*, quindi segmenti parziali e personalizzati del database.

#### 2. Livello concettuale (conceptual level)

- Rappresenta la *struttura logica globale* dell'intero database.
- Contiene entità, relazioni, attributi, vincoli, chiavi.

#### 3. Livello interno (physical level)

- Gestisce il *modo fisico* con cui i dati vengono memorizzati su disco.
- Include indici, strutture di accesso, pagine, file, tabelle fisiche.

### Obiettivo della separazione dei livelli

- Indipendenza dei dati
- Sicurezza (ogni utente vede solo ciò che serve)
- Portabilità e manutenibilità

---

### ➤ Componenti principali di un DBMS

#### 1. Gestore dell'archiviazione (Storage Manager)

- Organizza dati su file, pagine e blocchi.
- Comprende buffer manager, file manager, access methods.

#### 2. Query Processor

- Analizza, ottimizza ed esegue le query.
- Trasforma SQL in piano di esecuzione ottimizzato (optimizer).

#### 3. Gestore delle transazioni

- Garantisce ACID: Atomicità, Coerenza, Isolamento, Durabilità.
- Controlla la concorrenza (lock, multiversion concurrency control).

#### 4. Gestione sicurezza e autorizzazioni

- Controllo accessi, privilegi, auditing.

#### 5. Recovery System

- Ripristino in caso di crash tramite log, checkpoint, rollback/redo.

---

## 11. DBMS: definizione, componenti e funzioni

- Gestione dello storage
- Gestione della memoria e buffer
- Ottimizzazione delle query
- Controllo degli accessi
- Gestione transazioni
- Linguaggi: DDL, DML, DCL, TCL

---

## 12. Progettare basi di dati e modellare i dati

- Analisi requisiti
- Costruzione schema concettuale (diagramma ER)
- Passaggio al modello logico (tabelle, chiavi, relazioni)
- Modellazione fisica

---

## 13. Fondamenti di SQL

Sia teorici che pratici, con esercizi.

### DDL – Definizione dati

```
CREATE TABLE, ALTER TABLE, DROP TABLE
```

### DML – Manipolazione dati

```
SELECT, INSERT, UPDATE, DELETE
```

### Query fondamentali

- Filtri (WHERE)
- Ordinamenti (ORDER BY)
- Funzioni di aggregazione (SUM, COUNT, AVG...)
- GROUP BY, HAVING
- JOIN (INNER, LEFT, RIGHT, FULL)

### Query più avanzate

- Subquery
  - Funzioni sulle stringhe
  - Funzioni sulle date
-

## 14. Esercizi pratici SQL

- Creazione schema di esempio
  - Query di estrazione su più tabelle
  - Esempi di casi reali (vendite, prenotazioni, scolastico ecc.)
- 



## NO SQL E CLOUD

---

### □ 15. Database NoSQL

I database NoSQL (Not Only SQL) sono sistemi progettati per gestire grandi volumi di dati, non necessariamente strutturati, con alta scalabilità.

#### ➤ Perché sono nati

- Limitazioni della scalabilità dei DB relazionali in contesti Big Data
- Necessità di gestire dati eterogenei, distribuiti e ad alta velocità
- Architetture Cloud-Native e microservizi

#### ➤ Caratteristiche distintive

- **Schema-less** o schema flessibile
  - **Scalabilità orizzontale** (scaling-out)
  - **Elevata disponibilità** (replica distribuita)
  - **Performance in lettura/scrittura elevate**
  - Modelli dati più adatti a casi moderni (log, social, IoT, documenti JSON)
- 

#### ➤ Modelli principali NoSQL

##### 1. Document Store

- Dati in formato JSON/BSON
- Strutturati in documenti e collezioni
- Ideali per API REST e microservizi **Esempi:** MongoDB, CouchDB, Firestore

##### 2. Key-Value Store

- Dati come coppie chiave/valore
- Semplicità massima, performance altissime **Esempi:** Redis, DynamoDB

##### 3. Columnar / Wide-Column Store

- Basati su colonne invece che righe
- Ideali per data analytics e Big Data **Esempi:** Cassandra, HBase

##### 4. Graph Database

- Dati rappresentati tramite nodi e archi
  - Perfetti per reti sociali, recommendation, knowledge graph **Esempi:** Neo4j, ArangoDB, TigerGraph
- 

## ► Vantaggi e Svantaggi sintetici

### Vantaggi

- Scalabilità globale
- Elevata velocità
- Flessibilità nel modello dati
- Resistenza ai guasti distribuiti

### Svantaggi

- Meno standardizzazione
  - Consistenza eventuale (CAP theorem)
  - Mancanza di JOIN complessi
  - Non sempre adatti ad applicazioni ACID
- 

## □ 16. Cloud DB (Database sul Cloud)

Un **Cloud Database** è un database ospitato su un'infrastruttura Cloud pubblica, privata o ibrida, gestito come servizio.

## ► Perché usare un Cloud DB

- **Scalabilità automatica** (verticale/orizzontale)
  - **Affidabilità e replica geografica**
  - **Manutenzione ridotta** (backup, patching gestiti dal provider)
  - **Pay-per-use:** si paga solo ciò che si usa
  - Integrazione nativa con microservizi e architetture distribuite
- 

## ► Tipologie

### 1. Managed Relational Databases (SQL)

- Amazon RDS
- Azure SQL Database
- Google Cloud SQL
- Oracle Autonomous DB

### 2. NoSQL Cloud Databases

- DynamoDB
  - MongoDB Atlas
  - Google Firestore
  - Azure Cosmos DB
-

## ► Funzionalità chiave

- **Replica automatica** multi-zona
  - **Backup automatici** e point-in-time recovery
  - **Monitoraggio e scalabilità on-demand**
  - **Failover automatico**
  - **Sicurezza gestita**: crittografia, IAM, audit
- 

## ► Cloud-Native vs On-Premise

| Aspetto           | Cloud                | On-Premise                   |
|-------------------|----------------------|------------------------------|
| Scalabilità       | automatica           | complessa e costosa          |
| Costi             | variabili (OPEX)     | fissi (CAPEX)                |
| Sicurezza         | gestita dal provider | responsabilità interna       |
| Manutenzione      | minima               | elevata                      |
| Disaster Recovery | nativo               | da implementare internamente |

---

## ► Quando scegliere un Cloud DB

- Progetti che richiedono **alta scalabilità**
- Team con competenze DevOps/Cloud
- Applicazioni distribuite, IoT, mobile
- Sistemi con traffico variabile
- Start-up e PMI che vogliono evitare costi hardware